

Introduction to Algorithms and Data Structures

Chapter 1 of A. Levitin
Chapter 3 of A. Drozdek

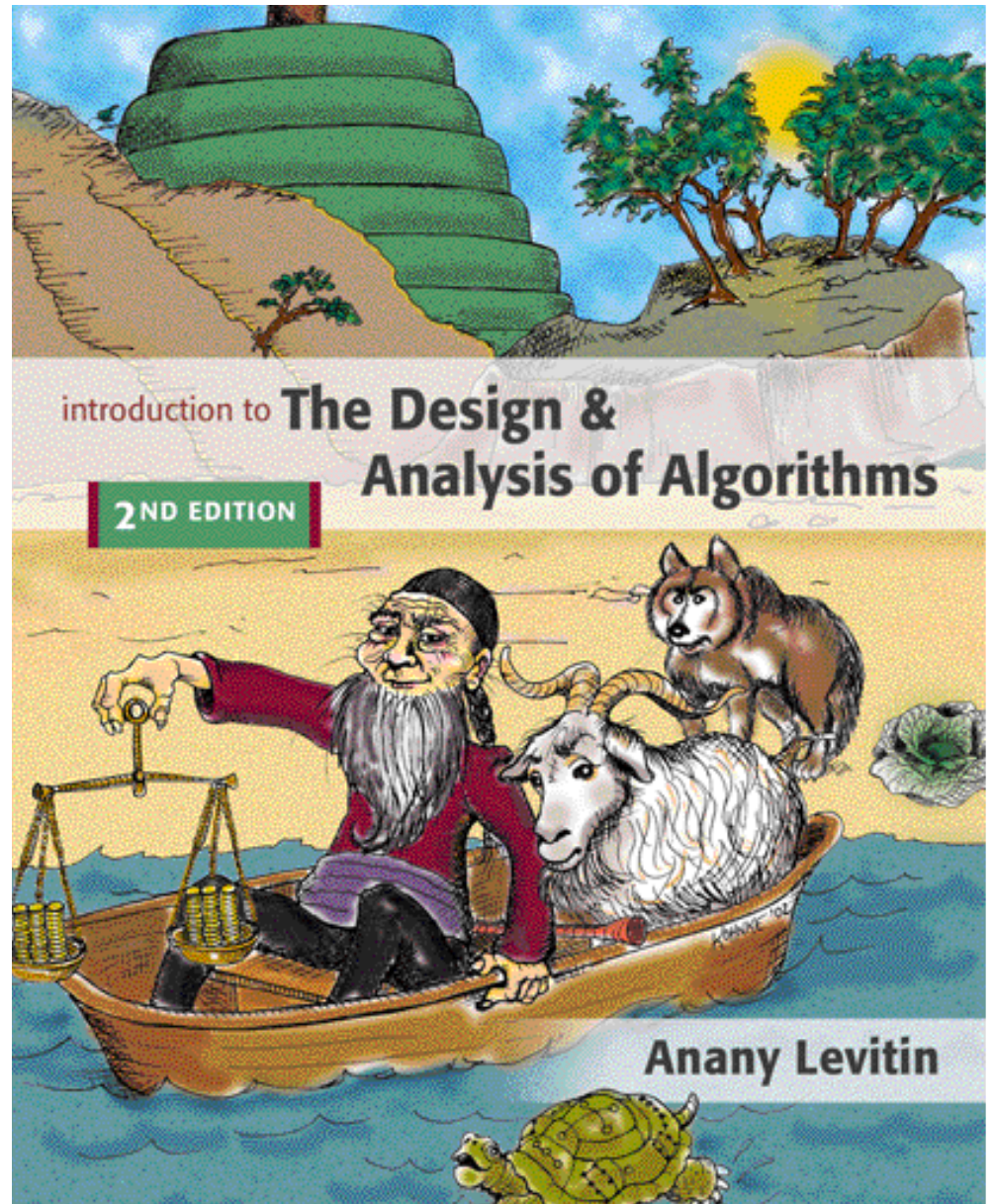
Objectives

Discuss the following topics:

- Introduction to algorithms
- Data Structures
 - Linked Lists

Chapter 1

Introduction to Algorithms



Why study algorithm?

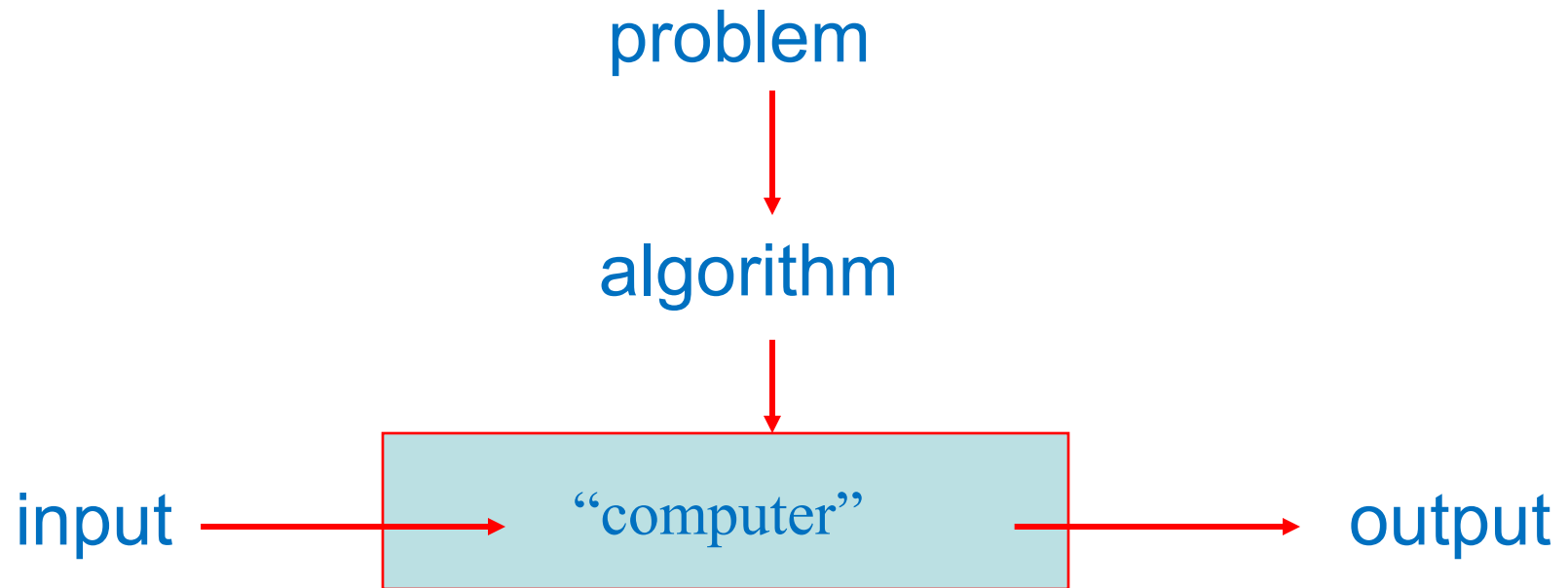
- “Algorithm is the **core of computer science**, and, in all fairness, can be said to be relevant to most of science, business, and technology.”
- “A person does not really understand something until after teaching it to someone else, actually, to a **computer**.”

What is an algorithm?

An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

“A person does not really understand something until after teaching it to someone else, actually, to a **computer.**”

Notion of algorithm



Algorithmic solution

Algorithm

- The nonambiguity requirement for each step
- Range of inputs for which an algorithm works has to be specified carefully.
- The same algorithms can be represented in several different ways.
- Several algorithms for solving the same problem may exist.
- Algorithms for the same problem can be based on very different ideas and can solve the problem with dramatically different speeds.

What is an algorithm?

- Recipe, process, method, technique, procedure, routine,... with following requirements:
 1. Finiteness
 - terminates after a finite number of steps
 2. Definiteness
 - rigorously and unambiguously specified
 3. Input
 - valid inputs are clearly specified
 4. Output
 - can be proved to produce the correct output given a valid input
 5. Effectiveness
 - steps are sufficiently simple and basic

Algorithm design strategies

- Brute force
- Greedy approach
- Divide and conquer
- Dynamic programming
- Decrease and conquer
- Backtracking and Branch and bound
- Transform and conquer
- Space and time tradeoffs

Analysis of Algorithms

- How good is the algorithm?
 - Correctness
 - Time efficiency
 - Space efficiency
- Does there exist a better algorithm?
 - Lower bounds
 - Optimality

Historical Perspective

- Euclid of Alexandria (Greek)
 - 3rd century B.C.
 - Euclid's algorithm for finding the Greatest Common Divisor (GCD)
- Muhammad ibn Musa al-Khwarizmi (Persian)
 - 9th century mathematician

www.lib.virginia.edu/science/parshall/khwariz.html



al-Khwarizmi -> Algorithm

- Al-Khwārizmī's method of solving linear and quadratic equations worked by first reducing the equation to one of six standard forms
 - squares equal roots ($ax^2 = bx$)
 - squares equal number ($ax^2 = c$)
 - roots equal number ($bx = c$)
 - squares and roots equal number ($ax^2 + bx = c$)
 - squares and number equal roots ($ax^2 + c = bx$)
 - roots and number equal squares ($bx + c = ax^2$)

Euclid's Algorithm

Problem: Find $\gcd(m,n)$, the greatest common divisor of two nonnegative, not both zero integers m and n

Examples: $\gcd(60,24) = 12$, $\gcd(60,0) = 60$,
 $\gcd(0,0) = ?$

Euclid's algorithm is based on repeated application of equality

$$\gcd(m,n) = \gcd(n, m \bmod n)$$

until the second number becomes 0, which makes the problem trivial.

Example: $\gcd(60,24) = \gcd(24,12) = \gcd(12,0) = 12$

Two descriptions of Euclid's algorithm

Step 1 If $n = 0$, return m and stop; otherwise go to Step 2

Step 2 Divide m by n and assign the value of the remainder to r

Step 3 Assign the value of n to m and the value of r to n .
Go to Step 1.

```
while  $n \neq 0$  do  
     $r \leftarrow m \bmod n$   
     $m \leftarrow n$   
     $n \leftarrow r$   
return  $m$ 
```

Other methods for computing $\gcd(m,n)$

Consecutive integer checking algorithm

Step 1 Assign the value of $\min\{m,n\}$ to t

Step 2 Divide m by t . If the remainder is 0, go to Step 3;
otherwise, go to Step 4

Step 3 Divide n by t . If the remainder is 0, return t and stop;
otherwise, go to Step 4

Step 4 Decrease t by 1 and go to Step 2

Other methods for $\gcd(m,n)$ [cont.]

Middle-school procedure

Step 1 Find the prime factorization of m

Step 2 Find the prime factorization of n

Step 3 Find all the common prime factors

Step 4 Compute the product of all the common
prime factors
and return it as $\gcd(m,n)$

Is this a legitimate algorithm?

		gcd(60, 24)			
1.1)	Find the biggest number that can divide both			for (i=2 -> 24) if(60 / i && 24 / i) gcd = i	
1.2)	Find the biggest number that can divide both			for (i=24 -> 2) if(60 / i && 24 / i) return i	gcd(19, 53)?
2)	High school			(60, 24) / 2 (30, 12) / 2 (15, 6) / 3 (5, 2) gcd = 2 x 2 x 3	
3)	Common Factors			60 = 2 x 2 x 3 x 5 24 = 2 x 2 x 2 x 3 gcd = 2 x 2 x 3 = 12	
4)	Euclid		gcd (60, 24)	= gcd (24, 60 mod 24) = gcd (24, 12) = gcd (12, 24 mod 12) = gcd (12, 0) = 12	

Sieve of Eratosthenes

ALGORITHM *Sieve*(n)

//Implements the sieve of Eratosthenes

//Input: An integer $n \geq 2$

//Output: Array L of all prime numbers less than or equal to n

for $p \leftarrow 2$ **to** n **do** $A[p] \leftarrow p$

for $p \leftarrow 2$ **to** $\lfloor \sqrt{n} \rfloor$ **do** //see note before pseudocode

if $A[p] \neq 0$ // p hasn't been eliminated on previous passes

$j \leftarrow p * p$

while $j \leq n$ **do**

$A[j] \leftarrow 0$ //mark element as eliminated

$j \leftarrow j + p$

//copy the remaining elements of A to array L of the primes

$i \leftarrow 0$

for $p \leftarrow 2$ **to** n **do**

if $A[p] \neq 0$

$L[i] \leftarrow A[p]$

$i \leftarrow i + 1$

return L

Example: 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Linked Lists

Discuss the following topics:

- Linked Lists
- Singly Linked Lists
- Doubly Linked Lists

Chapter 3 of Drozdek

Singly Linked Lists

- A **linked structure** is a collection of nodes storing data and links to other nodes
- A **linked list** is a data structure composed of nodes, each node holding some information and a reference to another node in the list
- A **singly linked list** is a node that has a link only to its successor in this sequence

Singly Linked Lists (continued)

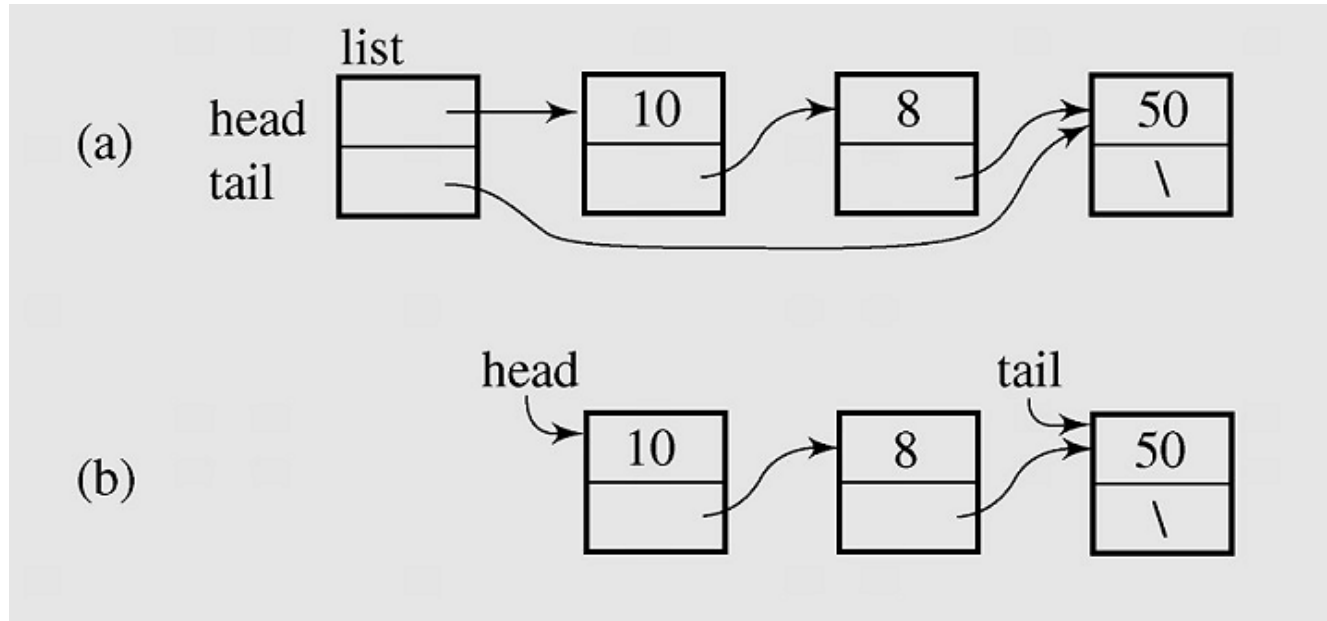


Figure 3-3 A singly linked list of integers

Singly Linked Lists (continued)

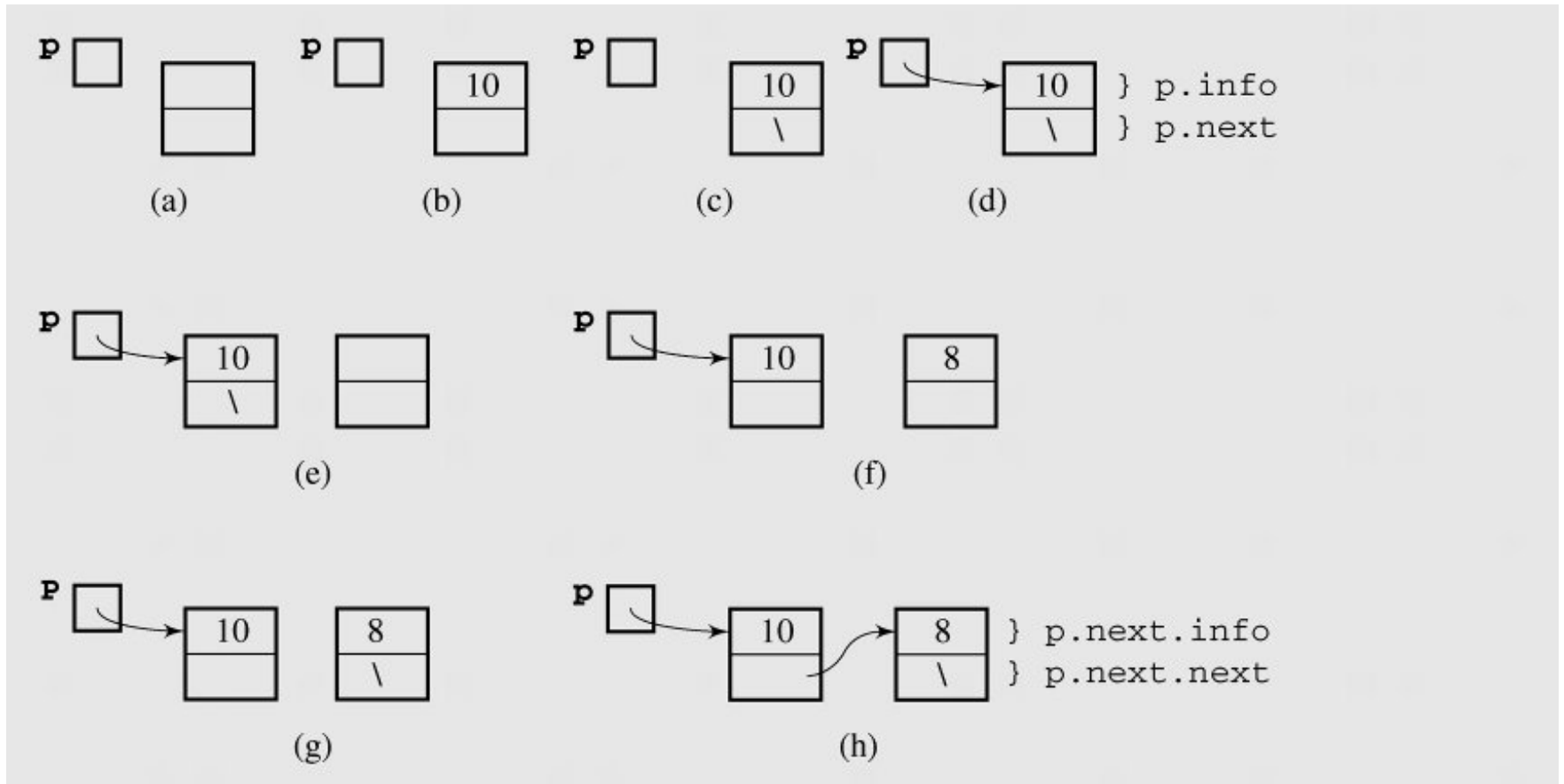


Figure 3-1 A singly linked list

Singly Linked Lists (continued)

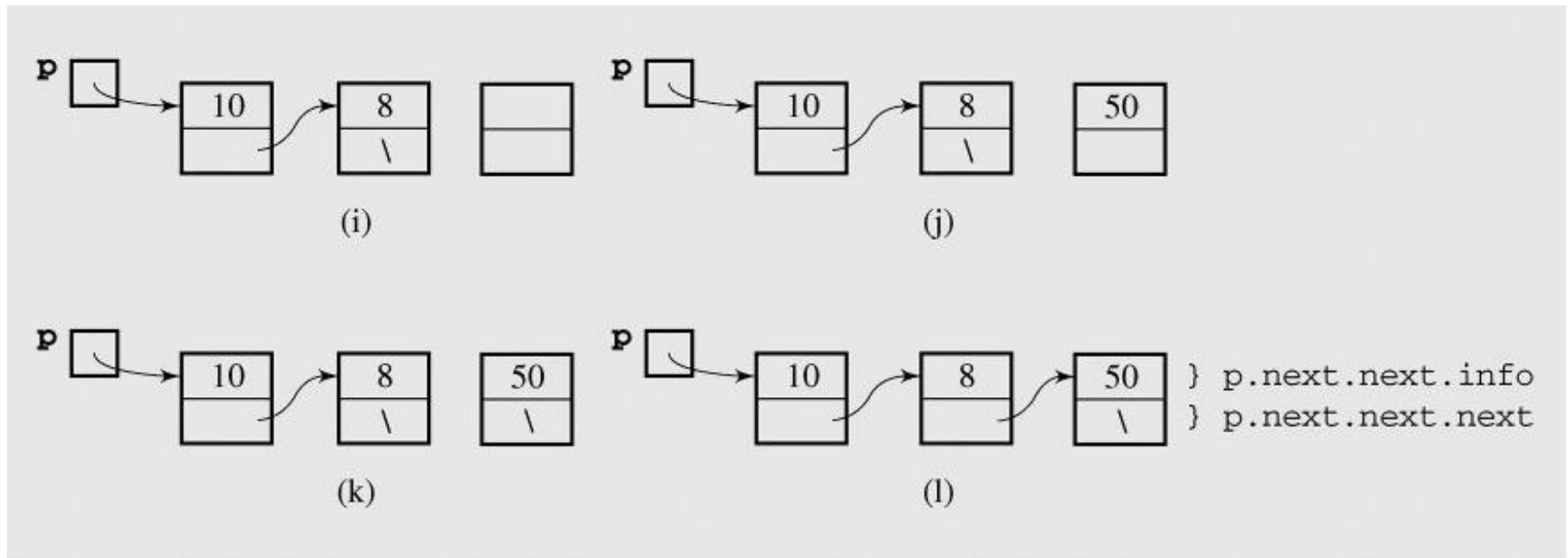


Figure 3-1 A singly linked list (continued)

Node of Singly Linked Lists

```
class LinkedListNode{  
    constructor(newData){  
        this.data = newData;  
        this.next = null;  
    }  
}
```


Singly Linked Lists (continued)

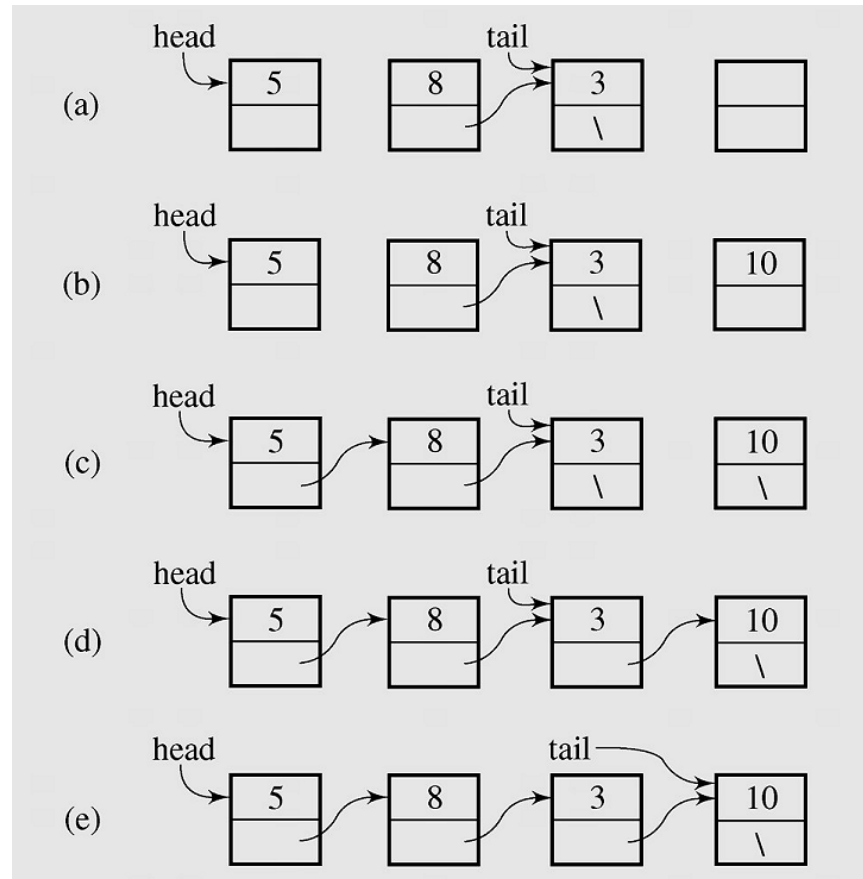


Figure 3-5 Inserting a new node at the end of a singly linked list

Singly Linked Lists

```
export class LinkedList {  
  constructor(){  
    this.head = this.tail = null;  
    this.size = 0;  
  }  
  isEmpty(){  
    return this.head==null;  
  }  
  addToTail(newElement){  
    var newNode = new LinkedListNode(newElement);  
    if(this.isEmpty()){  
      this.head = this.tail = newNode;  
    }else{  
      this.tail.next = newNode;  
      this.tail = newNode;  
    }  
    this.size++;  
  }  
}
```

Singly Linked Lists (continued)

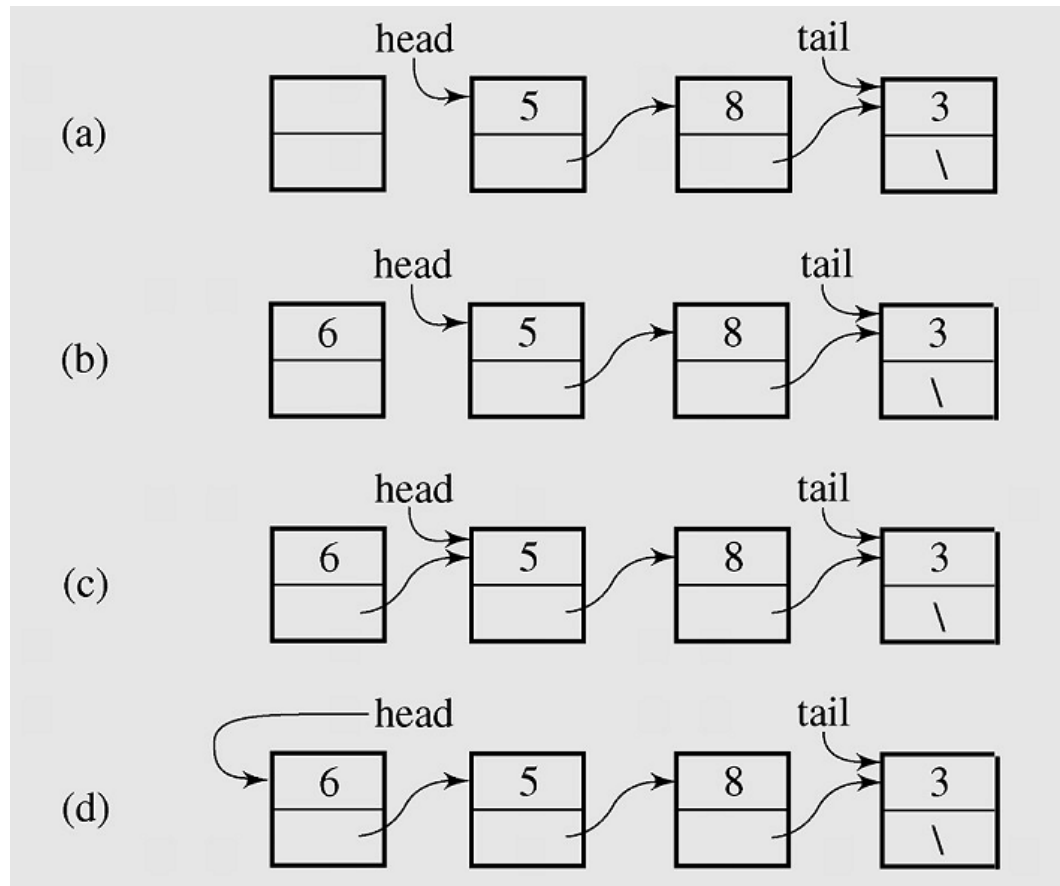


Figure 3-4 Inserting a new node at the beginning of a singly linked list

Singly Linked Lists (continued)

```
addToHead(newElement){  
    var newNode = new LinkedListNode(newElement);  
    if(this.isEmpty()){  
        this.head = this.tail = newNode;  
    }else{  
        newNode.next = this.head;  
        this.head = newNode;  
    }  
    this.size++;  
}
```

Singly Linked Lists (continued)

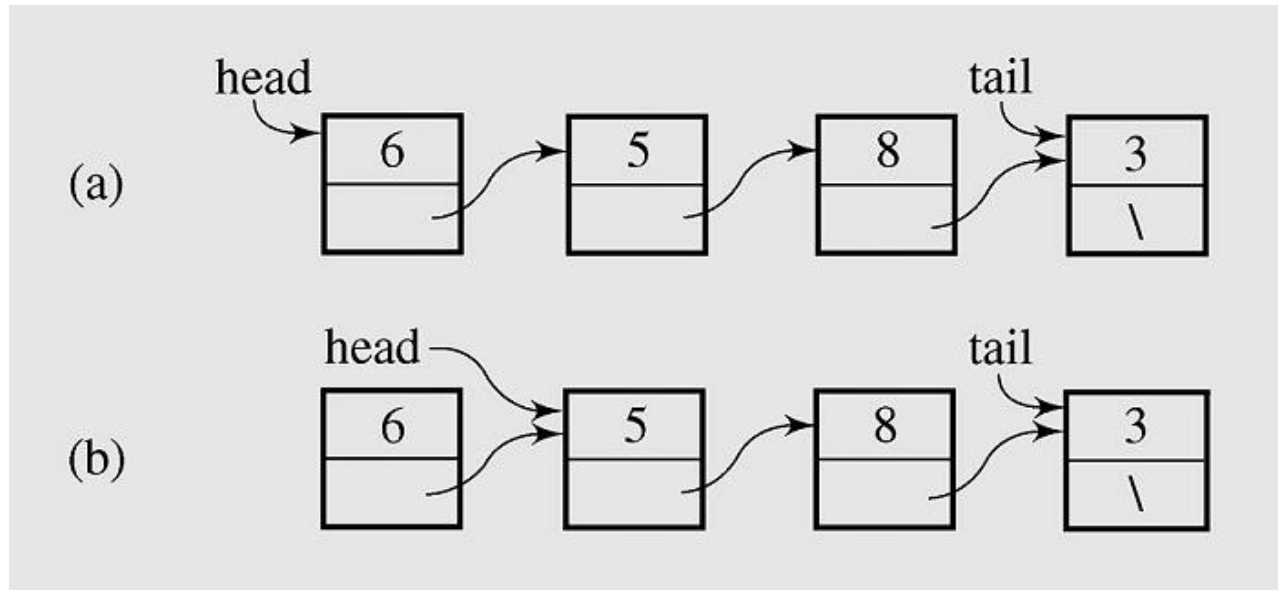


Figure 3-6 Deleting a node from the beginning of a singly linked list

Singly Linked Lists (continued)

```
removeFromHead(){  
  if (this.head == null){  
    console.log("There is no node to remove.");  
    return;  
  }  
  if( this.head.next == null){  
    this.head = this.tail = null;  
  }else{  
    this.head = this.head.next;  
  }  
  this.size--;  
}
```

Singly Linked Lists (continued)

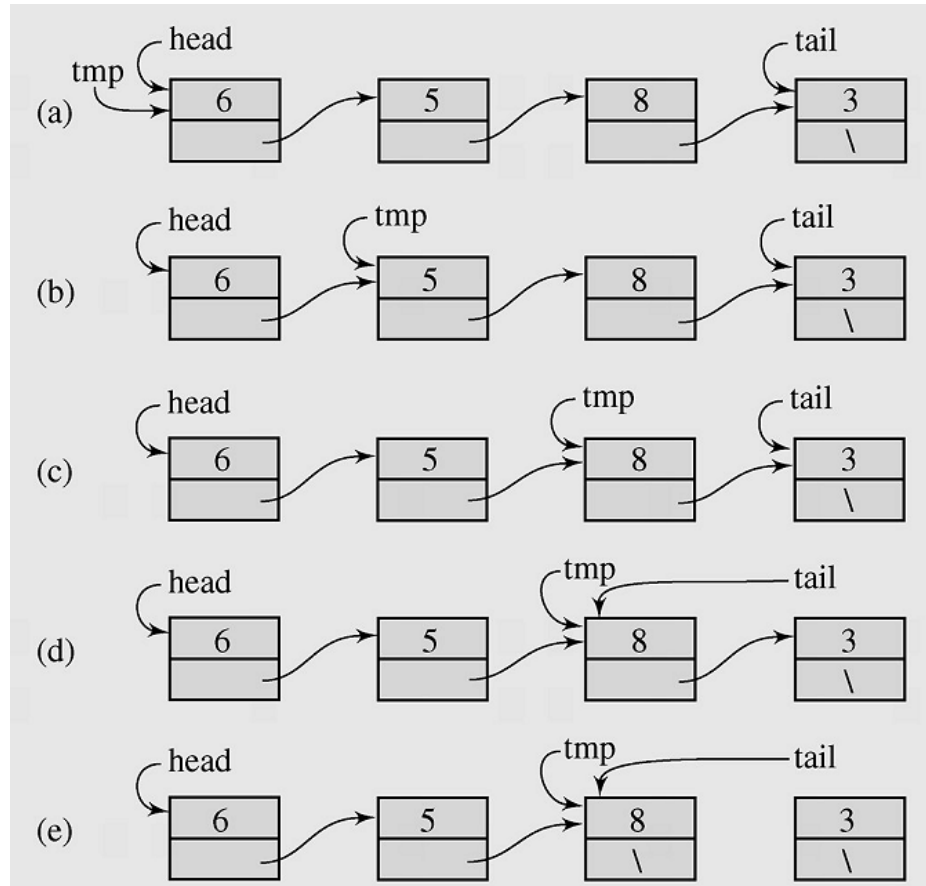


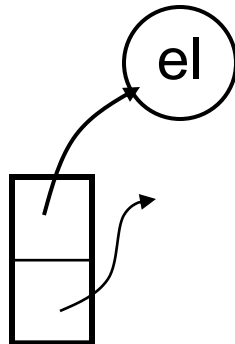
Figure 3-7 Deleting a node from the end of a singly linked list

Singly Linked Lists (continued)

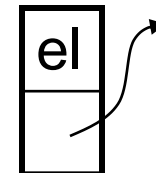
```
removeFromTail(){  
    if (this.head == this.tail){//empty list or only one node  
        this.head = this.tail = null;  
        console.log("No more node.");  
        return;  
    }else{  
        var temp;  
        for (temp = this.head; temp.next != this.tail; temp = temp.next){  
        }  
        this.tail = temp;  
        this.tail.next = null;  
        this.size--;  
    }  
}
```


Singly linked list node

```
public class SLLNode<T> {  
    public T info;  
    public SLLNode<T> next;  
    public SLLNode() {  
        next = null;  
    }  
    public SLLNode(T el) {  
        info = el; next = null;  
    }  
    public SLLNode(T el, SLLNode<T> p) {  
        info = el; next = p;  
    }  
}
```



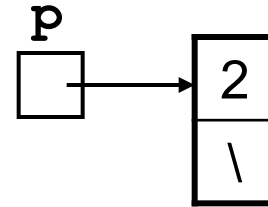
abbreviated as:



Singly linked list

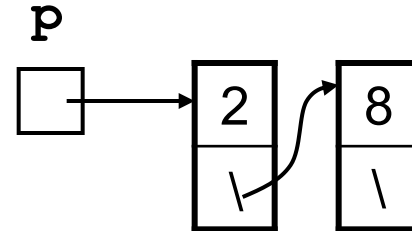
```
SLLNode<Integer> p;
```

```
p = new SLLNode<Integer>(2);
```



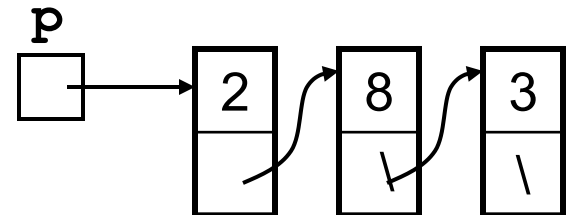
```
p.next =
```

```
new SLLNode<Integer>(8);
```



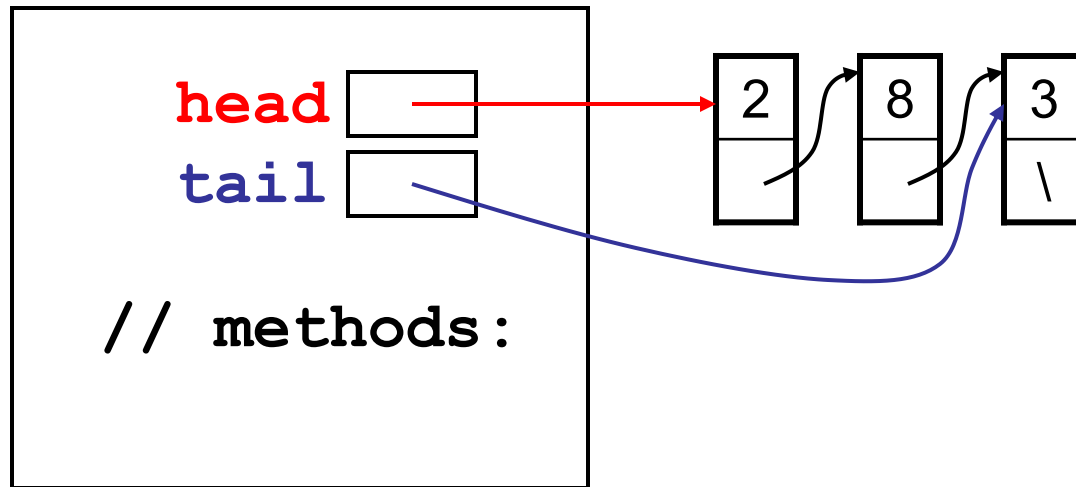
```
p.next.next =
```

```
new SLLNode<Integer>(3);
```

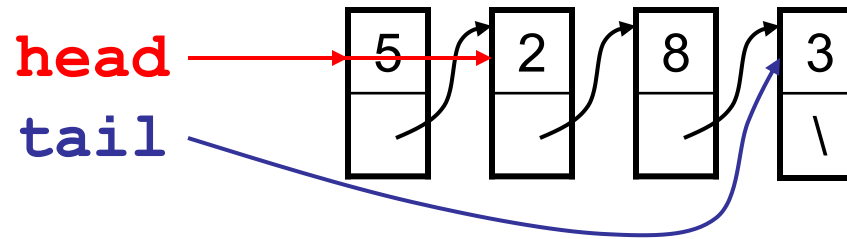


Singly linked list

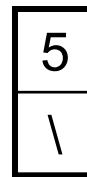
```
public class SLL<T> {  
    protected SLLNode<T> head, tail;  
    // methods:  
    .....  
}
```



Singly linked list: insertion at the head

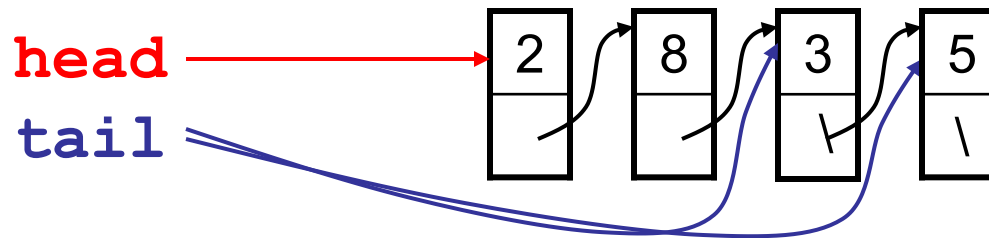


special case:

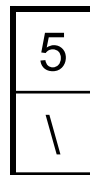


head → null
tail → null

Singly linked list: insertion at the tail

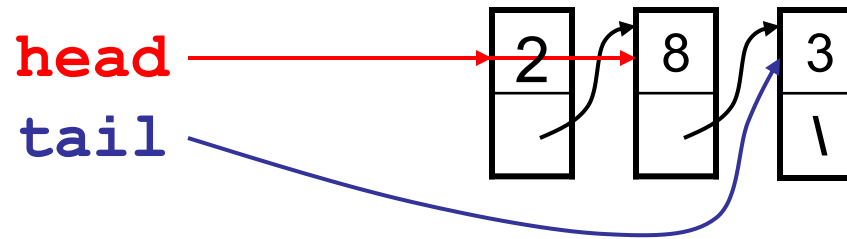


special case:

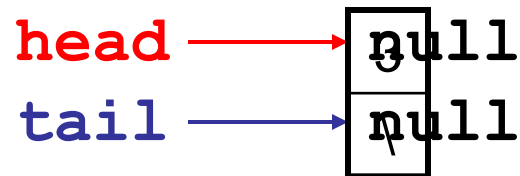


head → null
tail → null

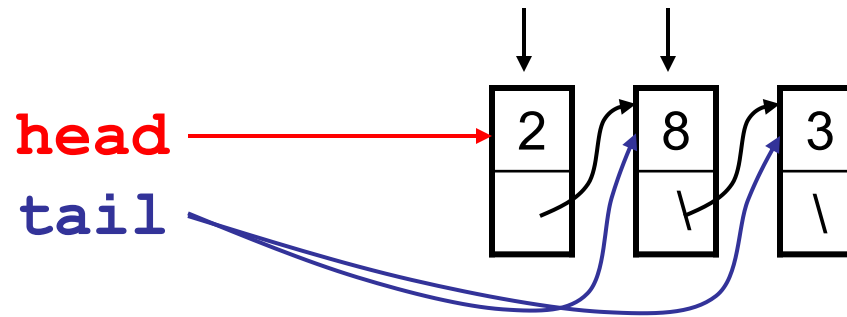
Singly linked list: deletion from the head



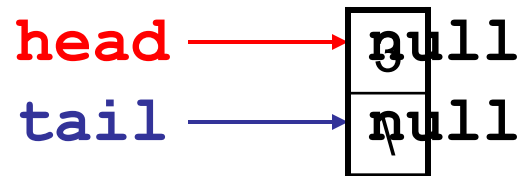
special case:



Singly linked list: deletion from the tail



special case:



Doubly Linked Lists

- A **doubly linked list** is when each node in a linked list has two reference fields, one to the successor and one to the predecessor

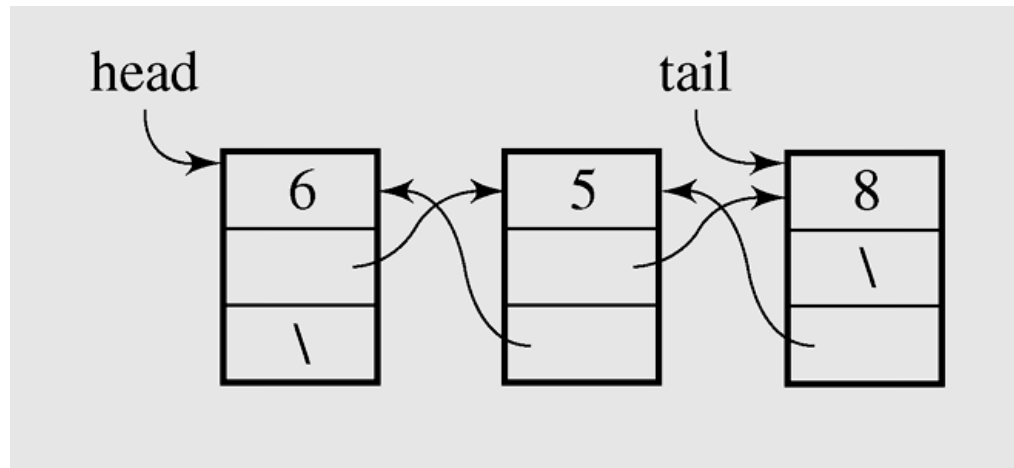


Figure 3-10 A doubly linked list

Doubly Linked Lists (continued)

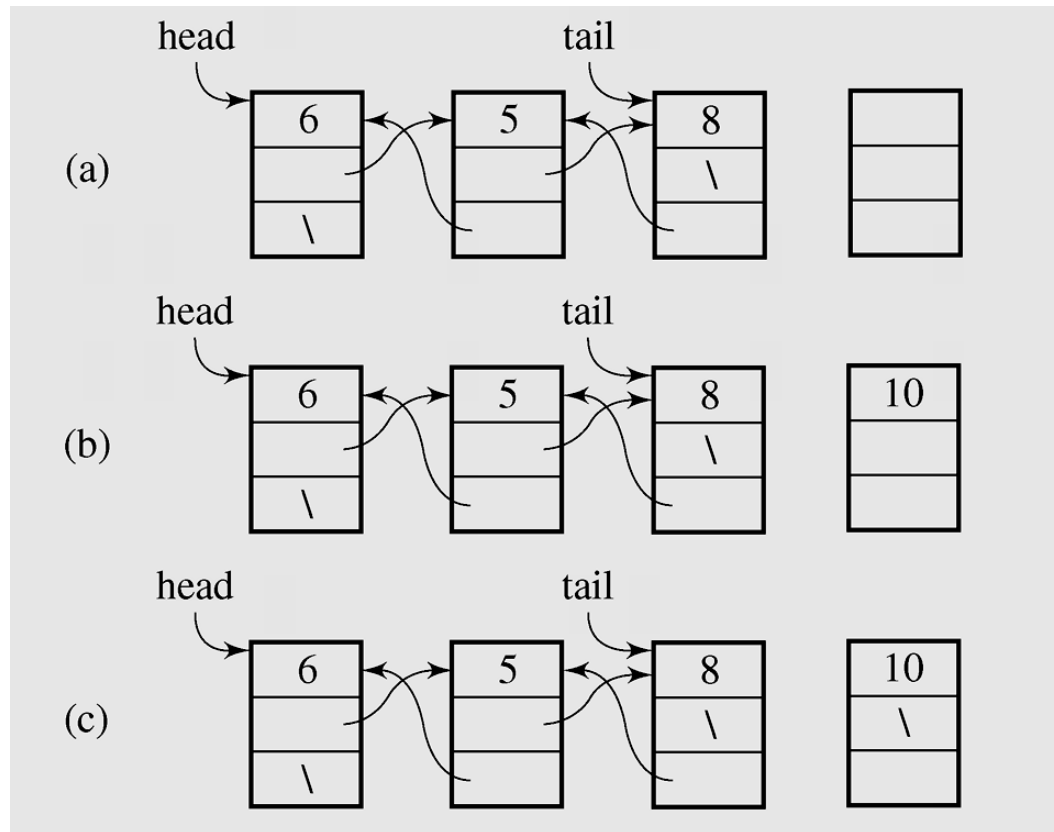


Figure 3-12 Adding a new node at the end of a doubly linked list

Doubly Linked Lists (continued)

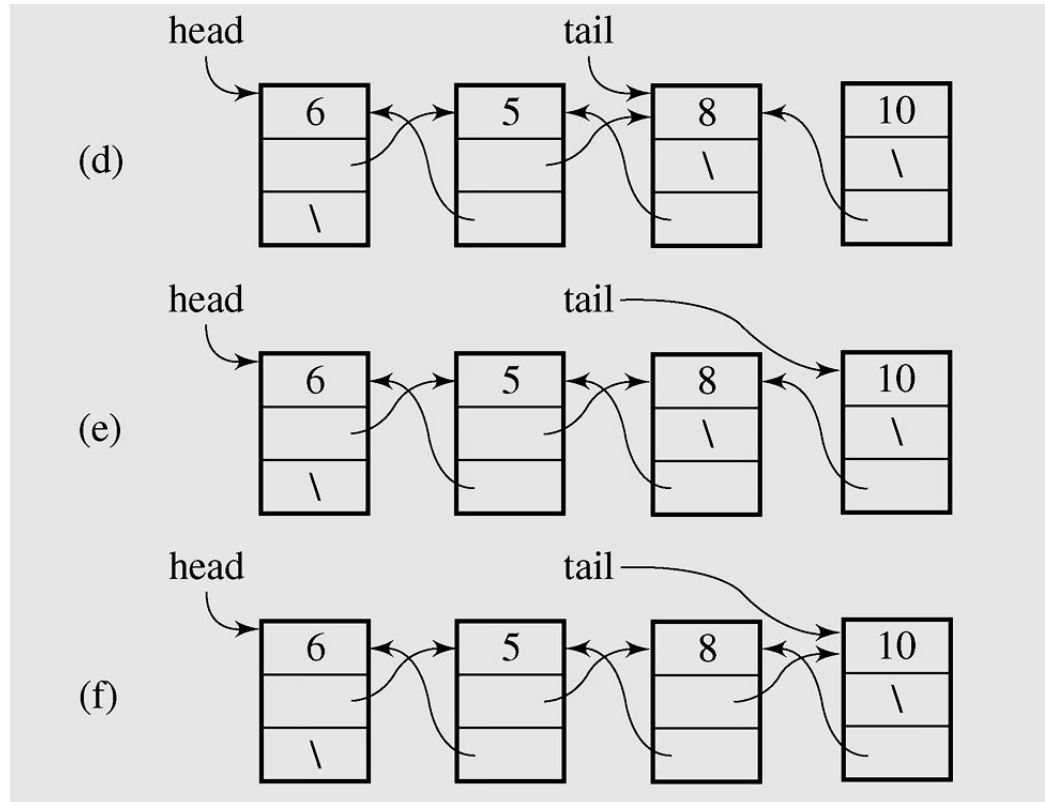


Figure 3-12 Adding a new node at the end of a doubly linked list (continued)

Doubly Linked Lists (continued)

```
public int removeFromTail() {  
    int el = tail.info;  
    if (head == tail)    // if only one node in the list;  
        head = tail = null;  
    else {                // if more than one node in the list;  
        tail = tail.prev;  
        tail.next = null;  
    }  
    return el;  
}  
.  
.  
.  
.  
.  
.  
.  
.  
.  
.  
}
```

Figure 3-11 An implementation of a doubly linked list (continued)

Doubly Linked Lists (continued)

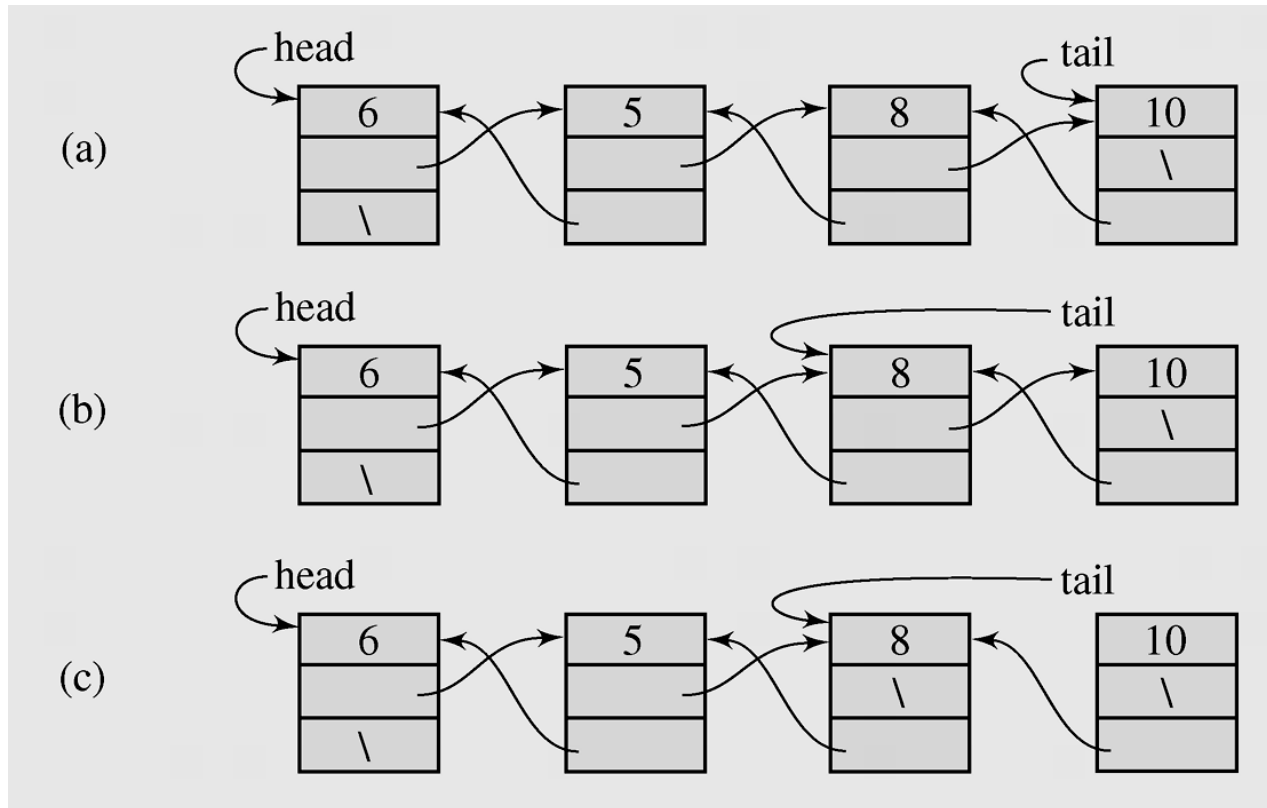
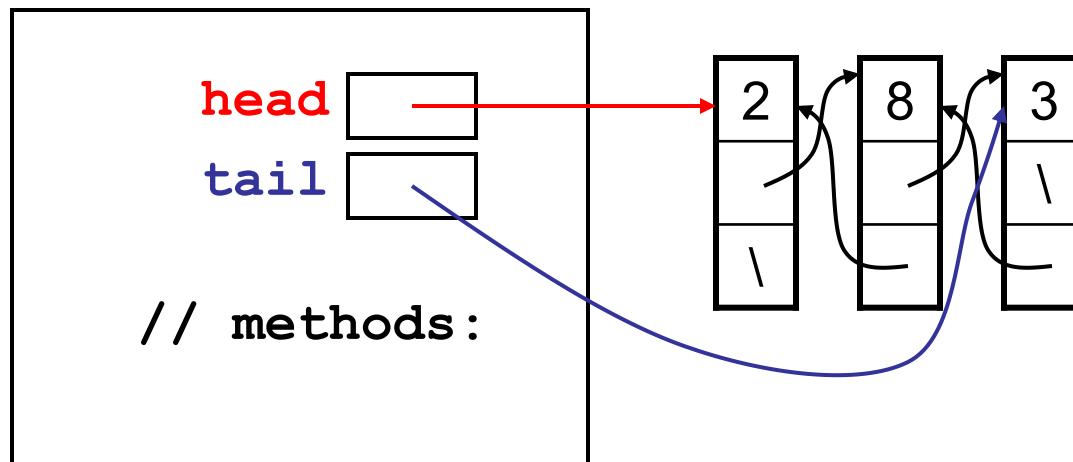


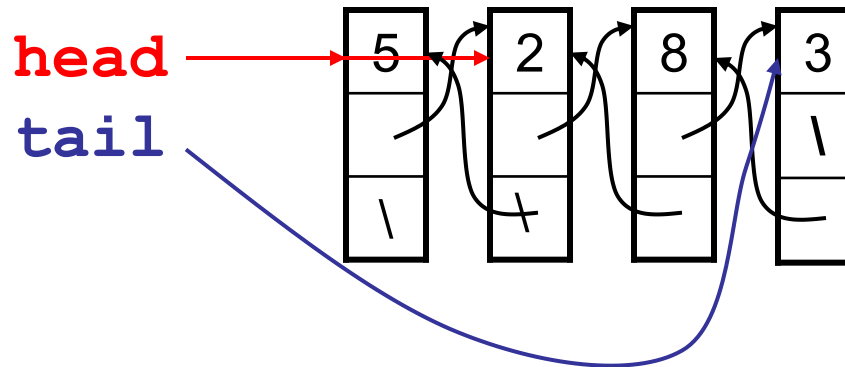
Figure 3-13 Deleting a node from the end of a doubly linked list

Doubly linked list

```
public class DLLNode<T> {  
    public T info;  
    public DLLNode<T> next, prev;  
    // constructors:  
    . . . . .  
}  
  
public class DLL<T> {  
    protected DLLNode<T> head, tail;  
    // methods:  
    . . . . .  
}
```



Doubly linked list: insertion at the head

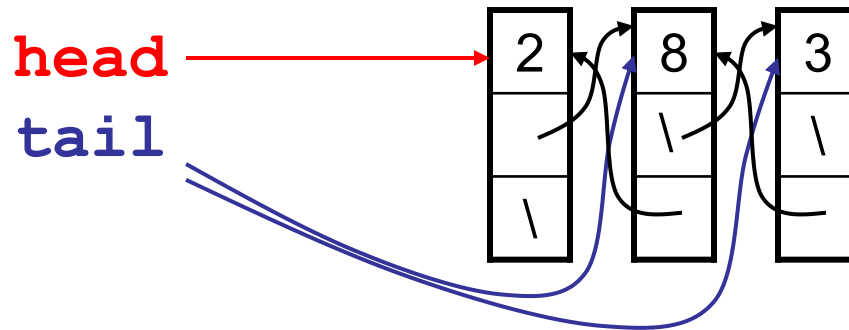


special case:

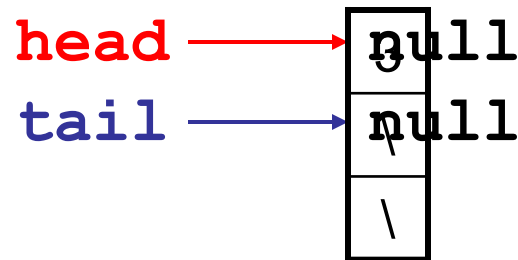


head → null
tail → null

Doubly linked list: deletion from the tail



special case:



Circular Lists

- A **circular list** is when nodes form a ring: The list is finite and each node has a successor

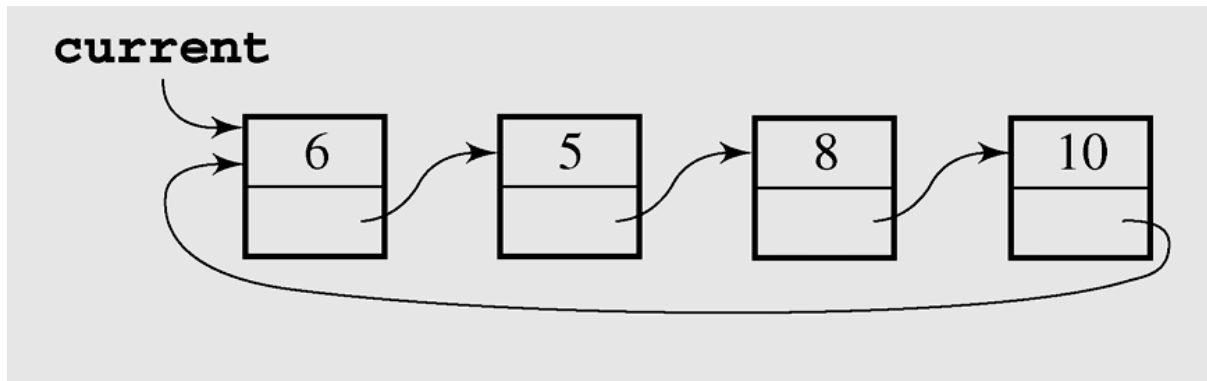


Figure 3-14 A circular singly linked list

Circular Lists (continued)

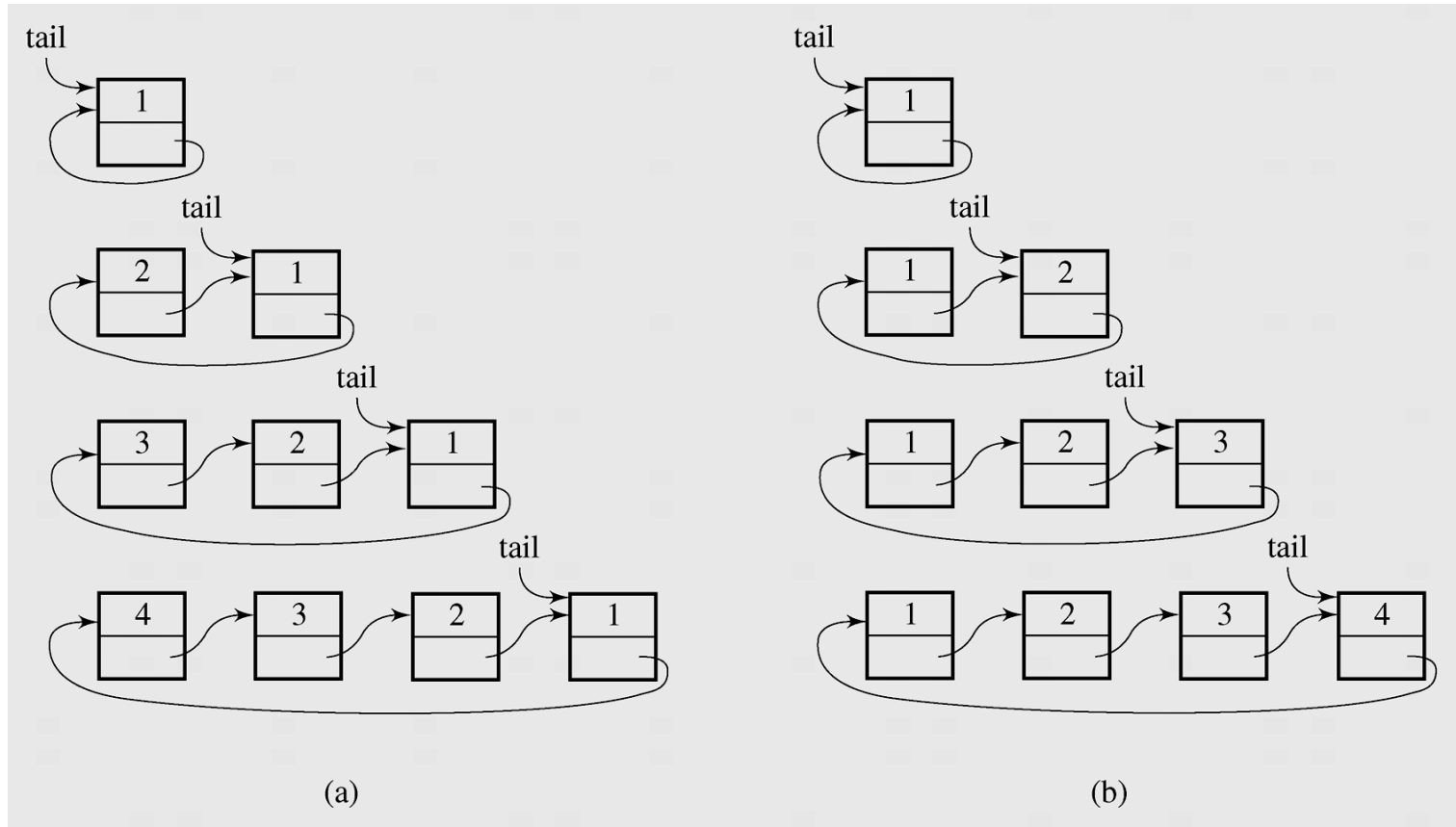


Figure 3-15 Inserting nodes at the front of a circular singly linked list (a) and at its end (b)

Circular Lists (continued)

- What is the source code of addToHead() method for Circular Singly Linked List?

```
public void addToTail ( int el){  
        ???  
}
```

- What's about addToHead (int el) method?

Circular Lists (continued)

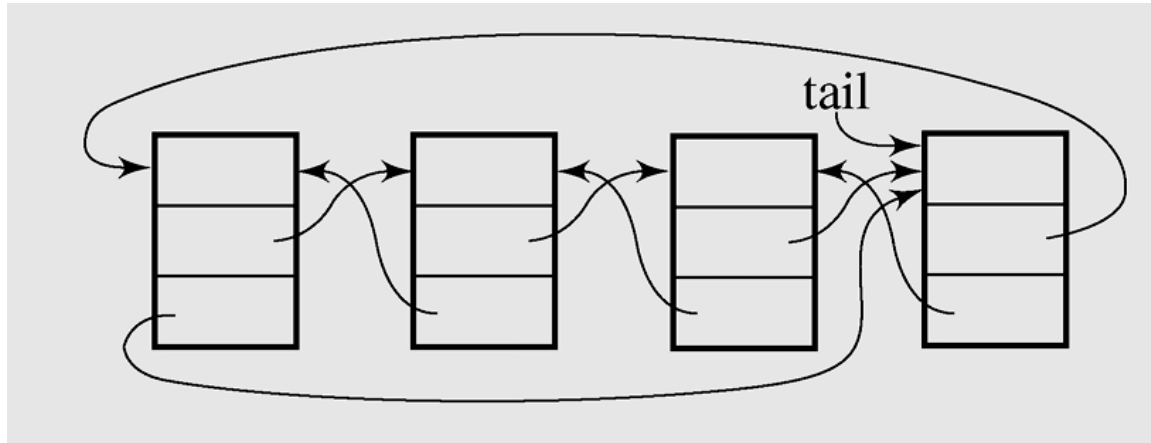
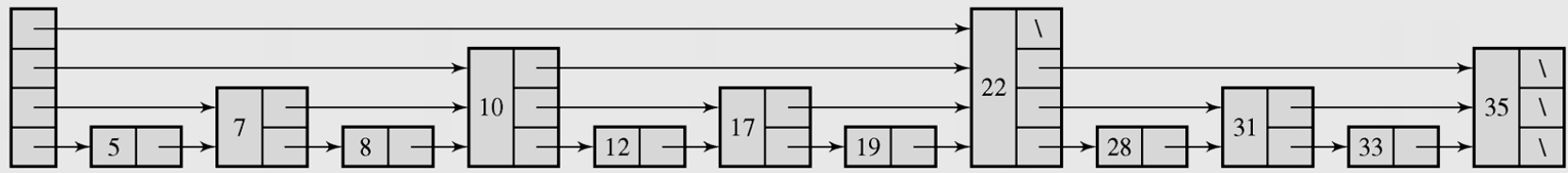


Figure 3-16 A circular doubly linked list

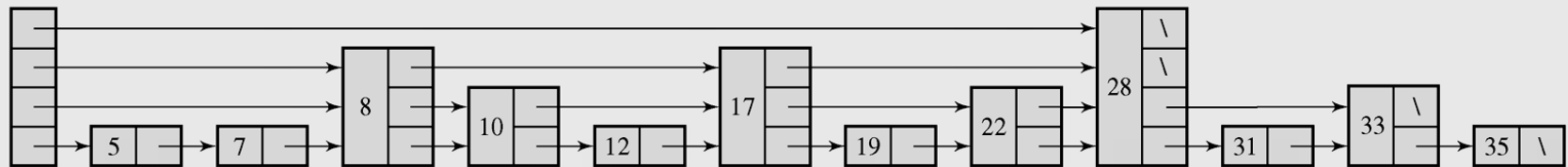
Skip Lists

- A **skip list** is a variant of the ordered linked list that makes a nonsequential search possible
- In a skip list of n nodes, for each k and i such that $1 \leq k \leq \lfloor \lg n \rfloor$ and $1 \leq i \leq \lfloor n/2^{k-1} \rfloor - 1$,
the node in position $2^{k-1} \cdot (i)$ **points to**
the node in position $2^{k-1} \cdot (i + 1)$
- The number of reference fields indicates the **level** of each node, and the number of levels is $maxLevel = \lfloor \lg n \rfloor + 1$

Skip Lists (continued)



(a)



(b)

Figure 3-17 A skip list with (a) evenly and (b) unevenly spaced nodes of different levels

Sparse Tables

- A **sparse table** refers to a table that is populated sparsely by data and most of its cells are empty
- With a sparse table, the table can be replaced by a system of linked lists

Sparse Tables (continued)

students		classes		gradeCodes	
0	Sheaver Geo	0	Anatomy/ Physiology	0	A
1	Weaver Henry	1	Introduction to Microbiology	1	A-
2	Shelton Mary	:		2	B+
:		30	Advanced Writing	3	B
404	Crawford William	31	Chaucer	4	B-
405	Lawson Earl	:		5	C+
:		115	Data Structures	6	C
5206	Fulton Jenny	116	Cryptology	7	C-
5207	Craft Donald	117	Computer Ethics	8	D
5208	Oates Key	:		9	F
:					

(a) (b) (c)

Figure 3-22 Arrays and sparse table used for storing student grades

Sparse Tables (continued)

		grades					student						
		0	1	2	...	404	405	...	5206	5207	5208	...	7999
class	0										3		
	1	1		4		7			1				
	:												
	30		5								3		
	31	0					5						
	:												
	115			0		4				5			
	116			3									
	117												
	:												
299													

(d)

Figure 3-22 Arrays and sparse table used for storing student grades (continued)

Sparse Tables (continued)

classesTaken1											
	0	1	2	...	404	405	...	5206	5207	5208	...7999
0	1	30	1		1	31		1	115	0	
1	31		115		115	64		33	121	30	
2	124		116		218	120		86	146	208	
3	136				221			121	156	211	
4					285			203		234	
5					292						
6											
7											

classesTaken2											
	0	1	2	...	404	405	...	5206	5207	5208	...7999
0	1	5	4		7	5		1	5	3	
1	0		0		4	5		1	5	3	
2	0		3		6	4		0	3	2	
3	2				5			2	0	3	
4					3			2		1	
5					3						
6											
7											

(a)

Figure 3-23 Two-dimensional arrays for storing student grades

Sparse Tables (continued)

studentsInClass1									
	0	1	...	30	31	...	115	116	... 299
0	5208	0		1	0		2	2	
1		2		5208	405		404		
2		404					5207		
3		5206							
...									
249									

studentsInClass2									
	0	1	...	30	31	...	115	116	... 299
0	3	0		5	0		0	3	
1		0		3	5		4		
2		7					5		
3		1							
...									
249									

(b)

Figure 3-23 Two-dimensional arrays for storing student grades (continued)

Sparse Tables (continued)

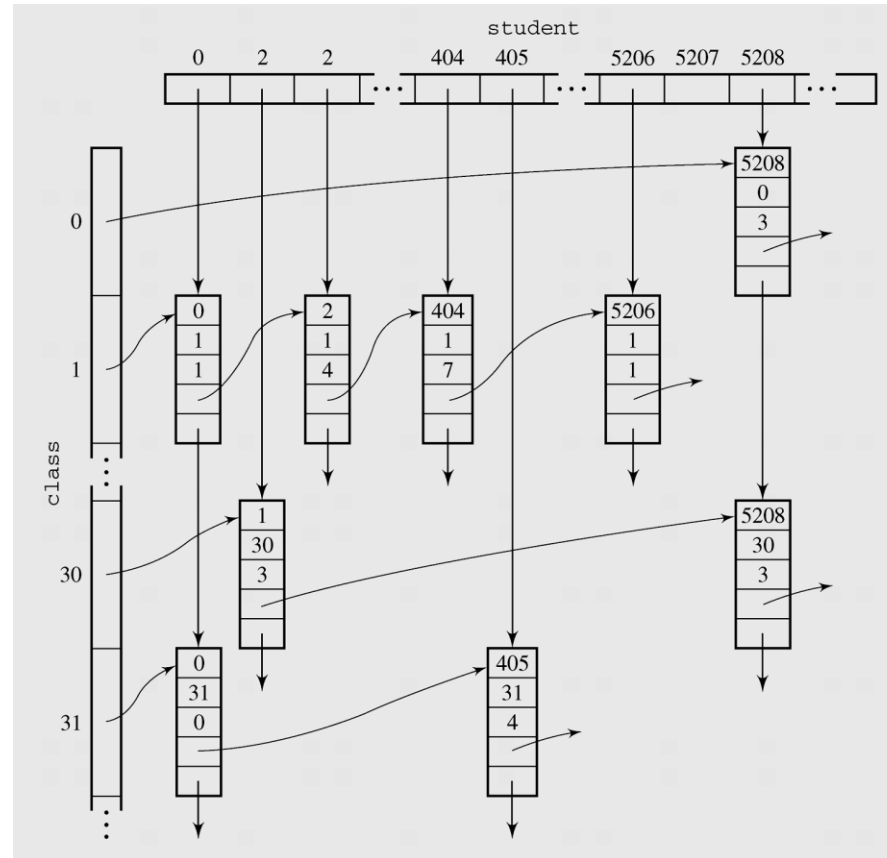


Figure 3-24 Student grades implemented using linked lists

Summary

- A linked structure is a collection of nodes storing data and links to other nodes.
- A linked list is a data structure composed of nodes, each node holding some information and a reference to another node in the list.
- A singly linked list is a node that has a link only to its successor in this sequence.
- A circular list is when nodes form a ring: The list is finite and each node has a successor.

Summary (continued)

- A skip list is a variant of the ordered linked list that makes a nonsequential search possible.
- There are four methods for organizing lists: move-to-front method, transpose method, count method, and ordering method.
- Optimal static ordering - all the data are already ordered by the frequency of their occurrence in the body of data so that the list is used only for searching, not for inserting new items.

Summary (continued)

- A sparse table refers to a table that is populated sparsely by data and most of its cells are empty.
- Linked lists allow easy insertion and deletion of information because such operations have a local impact on the list.
- The advantage of arrays over linked lists is that they allow random accessing.